

Sigma: An AERC Protocol-Based Multi-Agent Framework for Complex Engineering Design

Shidong Stone[‡]

May 2026

Abstract

Multi-agent AI systems promise to tackle complex engineering challenges beyond single-model capabilities, yet existing frameworks suffer from groupthink, hallucination cascades, and a lack of structured quality assurance. We present Sigma (Σ), a framework built around the AERC (Analyze $\rightarrow$ Execute $\rightarrow$ Review $\rightarrow$ Converge) protocol that orchestrates specialized engineering agents through structured deliberation cycles. Sigma introduces four key mechanisms: (1) blind independent parallel analysis with mandatory cross-review to catch errors before they propagate, (2) consensus estimation with calibrated confidence intervals when computational tools return unreliable data, (3) automatic complexity-adaptive tiering (LITE/STANDARD/RIGOROUS) that scales computational cost with task difficulty, and (4) automatic detection of massive LLM failures that would otherwise produce plausible-looking but factually empty outputs. We validate Sigma through a real-world engineering challenge: designing a deep energy retrofit for a 15,000 m² commercial office building, integrating DOE EnergyPlus whole-building simulation, thermal envelope analysis, and HVAC system modeling. Across multiple design iterations, Sigma’s cross-review mechanism caught an 18% heat pump performance overestimate—arising from manufacturer catalog data that ignored part-load degradation and distribution losses—that had gone undetected in single-agent and sequential multi-agent analysis. The framework is open-source (AGPLv3) and available as the `sigma-aerc` PyPI package.

1 Introduction

Large language models (LLMs) have demonstrated remarkable capabilities in code generation, data analysis, and creative tasks. However, complex engineering design requires more than any single model can provide: it demands domain-specific expertise across multiple disciplines, the ability to use specialized computational tools, and rigorous quality assurance to catch errors before they lead to costly failures.

Multi-agent frameworks such as CrewAI, AutoGen, and LangGraph offer orchestration primitives for coordinating multiple LLM-powered agents. Yet in practice, these frameworks lack three critical properties for engineering applications:

1. **Error propagation resistance.** When multiple agents share a common conversation context, a single hallucinated value can cascade through the entire system. Standard sequential agent execution provides no structural barrier against this.

*Correspondence: <https://github.com/maimai-dot/RocketFactory>. All code, data, and design history are publicly archived.

[‡]Independent Researcher, RocketFactory Project. The `sigma-aerc` package (v0.1.0, 985 tests, AGPLv3) is available at <https://pypi.org/project/sigma-aerc/>.

2. **Adversarial quality assurance.** Engineering design requires someone to explicitly argue the “worst case” perspective. Without a designated devil’s advocate, multi-agent systems tend toward consensus too quickly—a phenomenon we term *groupthink convergence*, drawing on the classic group decision-making pathology identified by Janis [7].
3. **Cost-adaptive rigor.** Not all tasks require the same level of scrutiny. A trivia question needs one agent; a building energy retrofit design needs full adversarial review. Fixed orchestration topologies waste either compute or safety margin.

Sigma (Σ) addresses these gaps with a protocol-based approach. Rather than prescribing a fixed agent topology, Σ defines the AERC protocol—a structured deliberation cycle that any set of specialized agents can participate in. The protocol enforces independent parallel analysis (agents cannot see each other’s outputs during the Analysis phase), mandatory cross-review, and physics-based impossibility checks before convergence is declared.

The framework originated from a concrete engineering need: designing a deep energy retrofit for a large commercial building using an AI agent team, constrained to validated simulation tools and realistic budget limits. This mission, called GreenBuild, serves both as Σ ’s primary validation case and as a stress test for whether structured multi-agent protocols can produce engineering-quality outputs under real-world constraints.

2 The AERC Protocol

The AERC protocol structures each deliberation round into four sequential phases. All agents share a common *context state* (the task specification, prior results, and tool outputs) but are *information-isolated* during critical phases to prevent premature consensus.

Phase	Action	Isolation	Output
A (Analyze)	Independent analysis by each agent	Blind (agents cannot see peers)	Per-agent analysis
E (Execute)	Parallel tool invocation	Shared context	Tool results
R (Review)	Cross-review + devil’s advocate	Pairwise review	Review verdicts
C (Converge)	Convergence check + consensus est.	Open deliberation	Consensus or next round

Figure 1: The AERC protocol cycle. Each round produces either convergence (task complete) or a refined context for the next round.

2.1 Phase A: Analyze

Given the current context state S_t at round t , each agent $A_i \in \{A_1, \dots, A_n\}$ independently produces an analysis α_i^t conditioned *only* on S_t and the agent’s own domain expertise (role configuration θ_i , skill files $\mathcal{K}_i$). The defining constraint is:

$$\alpha_i^t \perp \alpha_j^t \quad \text{for all } j \neq i \text{ during Phase A} \quad (1)$$

That is, each agent’s analysis is structurally independent of peer analyses within the same round—the architectural isolation (analyses are collected before any are distributed) prevents any information flow between agents during Phase A. This is the defense against groupthink: an agent cannot lazily agree with another’s analysis because it cannot see it. Because the independence is enforced architecturally rather than through prompting, it is robust to prompt injection and agent non-compliance. We use the notation $\perp$ to denote this enforced isolation, distinct from statistical independence (which would additionally require uncorrelated outputs from the shared LLM backend).

In the GreenBuild case, 8 agents participate in a RIGOROUS-tier round: Project Architect (systems integration), HVAC Engineer (heating, cooling, ventilation), Building Physicist (thermal envelope, moisture, daylighting), Controls Engineer (building automation and controls), Energy Modeler (EnergyPlus/OpenStudio simulation), Cost Estimator (equipment, labor, lifecycle cost), Code Reviewer (ASHRAE/IECC compliance, devil’s advocate), and Research Analyst (technology scan, knowledge gap detection).

2.2 Phase E: Execute

Agents invoke domain-specific computational tools. Sigma uses OpenAI-compatible function calling as the primary mechanism, with a text-marker fallback for models that do not support native tool use. Tool results are appended to the shared context S_t and are immediately visible to all agents.

Key tools in the GreenBuild deployment include:

- **energyplus_simulator**: DOE EnergyPlus whole-building energy simulation (annual hourly timestep, multi-zone thermal balance)
- **envelope_modeler**: Thermal envelope analysis (U-value, thermal bridging, condensation risk, glazing solar heat gain)
- **openstudio_runner**: OpenStudio SDK for parametric building geometry, HVAC system configuration, and retrofit measure packaging
- **web_search**: Four-source search with credibility scoring

2.3 Phase R: Review

The Review phase introduces two structural quality mechanisms:

Cross-review. Each agent’s Phase A analysis α_i^t is reviewed by a different agent A_j ($j \neq i$). The reviewer checks for factual errors, inconsistent assumptions, and overlooked constraints. This pairwise review generates a verdict $\rho_{j \rightarrow i}^t \in \{\text{agree, disagree, revise}\}$ with specific evidence.

Devil’s advocate. The Code Reviewer role carries a mandatory worst-case review. For every design proposal, the Code Reviewer must identify at least one credible failure mode and argue from first principles why it could occur. The Code Reviewer holds veto power: a single well-reasoned compliance or performance objection can block convergence, forcing an additional round.

This mechanism proved essential in practice. As detailed in Section 5.2, the cross-review mechanism, initiated by the Code Reviewer’s devil’s advocate inquiry, caught an 18% heat pump performance overestimate—an error that no single agent flagged during independent analysis, and one that would have cascaded through a sequential agent pipeline undetected. The correction had significant implications for equipment sizing, supplemental heating capacity, and the project’s energy payback calculation.

2.4 Phase C: Converge

The Converge phase evaluates whether the multi-agent system has reached a stable consensus. Three criteria must all be satisfied:

1. **Numerical convergence.** Key quantitative estimates (annual energy use intensity, heating/cooling capacity, capital cost, payback period) vary by less than 5% between consecutive rounds.
2. **Qualitative convergence.** No new design conflicts or disagreements emerge in the Review phase.

3. **Physics sanity check.** All numerical values pass a rule-based physics impossibility filter (e.g., $\text{COP} > 0$, $\text{EUI} > 0$, heating capacity $>$ design heat loss, U-values within physically possible range, chiller COP above thermodynamic limit).

If all criteria are met, the system produces a final `REPORT.md` with consensus estimates and confidence intervals, plus a structured `result.json` for downstream tooling.

If convergence is not reached after 4 rounds (RIGOROUS tier) or 3 rounds (STANDARD tier), the system halts with a detailed non-convergence report. An oscillation detector monitors for cycles where agents alternate between two positions without progress—if detected, the Project Architect is prompted to break the tie with explicit reasoning.

3 Key Mechanisms

3.1 Cross-Review Architecture

The cross-review mechanism is the core defense against hallucination propagation. Its effectiveness depends on two design choices:

Blind initial analysis. Agents produce α_i^t without access to peer analyses. This prevents *anchoring*—the well-documented cognitive bias where initial information disproportionately influences subsequent judgments, even when that information is incorrect [6].

Domain-diverse reviewer assignment. Reviewers are assigned to maximize domain distance. The HVAC Engineer reviews the Building Physicist’s analysis (not the Energy Modeler’s), ensuring that cross-review catches errors a same-domain reviewer might share as blind spots.

The isolation trade-off. Information isolation is not cost-free. When agents cannot see each other’s analyses during Phase A, they cannot build on correct insights discovered by peers—each agent must independently derive conclusions that a collaborative approach could distribute across the team. This represents a deliberate *redundancy-over-efficiency* design choice: Sigma invests additional compute to generate independent perspectives, accepting that some agents will duplicate analytical work. The trade-off is justified when the cost of an undetected error (e.g., undersized heating equipment leading to occupant discomfort and tenant loss) exceeds the marginal cost of redundant computation by orders of magnitude. For tasks where errors are cheap and speed dominates—such as routine data transformation—the LITE tier disables isolation entirely, allowing direct execution with minimal overhead.

Empirical observation from the GreenBuild design campaign: cross-review caught an average of 1.7 qualitative inconsistencies per RIGOROUS round (31 issues across 18 rounds). To estimate what fraction of these would escape a sequential pipeline, we re-ran a subset of early design rounds in a sequential-review configuration where agents received prior agents’ full outputs. In this sequential mode, 11 of the 31 issues (approximately one-third) were not identified by any agent, as later agents were anchored by earlier incorrect outputs. This suggests that the redundancy cost (8 agents producing independent analyses) yields a meaningful error-detection premium for safety-critical engineering tasks.

3.2 Consensus Estimation

When computational tools return unreliable or simulated data (common in early-stage design where exact parameters are unknown), Σ switches to a structured consensus estimation protocol:

1. Each relevant agent independently estimates the target quantity (e.g., baseline building energy use intensity) based on their domain knowledge.
2. Agents then review each other’s estimates, providing specific reasoning for disagreements.

3. A consensus range $[v_{\text{low}}, v_{\text{high}}]$ is computed as the 20%-trimmed mean $\pm 1.5 \times$ the interquartile range of agent estimates. The confidence level $\in \{\text{LOW}, \text{MEDIUM}, \text{HIGH}\}$ is assigned based on the coefficient of variation (σ/μ) of the trimmed estimates and the independently assessed reliability of any tool data used as input.

This mechanism was validated in an early design iteration where the baseline annual energy use intensity (EUI) was estimated at $185 \pm 22 \text{ kWh/m}^2$ (MEDIUM confidence) from 4 independent agent estimates, before detailed EnergyPlus modeling became available. Subsequent EnergyPlus-based analysis produced divergent EUI estimates depending on weather file selection and internal load assumptions, illustrating why consensus estimation includes confidence calibration—when independent methods disagree substantially, the MEDIUM or LOW confidence rating signals that further data collection is needed.

3.3 Complexity-Adaptive Tiering

Not all tasks require the full 8-agent RIGOROUS protocol. Σ uses a pure rule-based complexity scorer (no LLM dependency) that evaluates task descriptions across 10 dimensions, each contributing 0–1 points:

Dimension	Weight	Scoring Rule
Domain count	1.0	1 pt per distinct discipline beyond the first (≥ 4 disciplines = 1.0)
Safety criticality	1.0	0.25 (informational), 0.5 (financial), 0.75 (reputational), 1.0 (physical)
Tool dependency	1.0	0.25 per required external tool (max 1.0 at ≥ 4 tools)
Novelty	1.0	0 (well-documented), 0.5 (partial precedent), 1.0 (no known precedent)
Constraint density	1.0	0.25 per hard constraint beyond 2 (budget, timeline, materials, regulations)
Interdependency	1.0	0 (independent sub-problems), 0.5 (sequential), 1.0 (tightly coupled)
Precision requirement	1.0	0 (order-of-magnitude), 0.5 ($\pm 20\%$), 1.0 ($\pm 5\%$ or tighter)
Information ambiguity	1.0	0 (all parameters known), 0.5 (partial data), 1.0 (data from unreliable sources)
Reversibility	1.0	0 (easily undone), 0.5 (reversible with effort), 1.0 (irreversible)
Stakeholder count	1.0	0.25 per distinct perspective beyond 2 (engineering, procurement, safety, etc.)

Table 1: Complexity scoring dimensions. Total score $c = \sum d_i \in [0, 10]$ determines tier selection (Table 2).

The dimension weights are uniform (1.0 each), reflecting a deliberate design choice: we treat domain breadth and safety criticality as equally informative for determining review rigor. The scoring rules use coarse bins (0, 0.25, 0.5, 0.75, 1.0) rather than fine-grained values to reduce sensitivity to minor wording differences in task descriptions. The total score $c \in [0, 10]$ maps to three tiers:

Tier	Score	Agents	Max Rounds	Mechanisms
LITE	$c \leq 2.5$	Director + 2–3	1	None (direct execution)
STANDARD	$c \leq 6.0$	6 core agents	3	Cross-review
RIGOROUS	$c > 6.0$	All 8 agents	4	Full: cross-review + devil’s advocate + consensus

Table 2: Complexity-adaptive tiers. A trivia question ($c \approx 0.5$) uses LITE; a building energy retrofit design ($c \approx 7.4$) uses RIGOROUS.

This adaptive system was validated in a controlled comparison: a LITE-tier trivia task completed in 1 round with 2 agents at a cost of **¥0.03** ($\sim \$0.004$), while a RIGOROUS building energy analysis required 3 rounds with 8 agents at **¥2.47** ($\sim \$0.34$)—roughly two orders of magnitude cost difference for commensurate differences in task complexity and safety requirements.

We note that the scoring weights and tier boundaries have been calibrated against the GreenBuild campaign only; systematic validation of the scoring function across multiple independent domains remains future work.

3.4 Massive LLM Failure Detection

After the Execute phase, Σ automatically checks whether $\geq 50\%$ of agents returned error responses. An “error response” is defined as any agent output that either (a) contains an explicit error marker from the LLM provider (API error, content filter refusal, token limit truncation), or (b) fails the minimum verbosity check (< 20 tokens of analysis content after stripping tool invocations). If the threshold is crossed, the round is immediately aborted (verdict = **error**) and the orchestrator reports which agents failed and at what rate. This prevents a dangerous failure mode observed in earlier iterations: when multiple agents silently produce empty or near-empty responses (due to API degradation, context window exhaustion, or content filtering), the remaining agents’ outputs can appear to “converge” on plausible values that are in fact unsupported by the full agent panel. The 50% threshold was chosen empirically—a single agent failure is survivable (cross-review catches it), but losing half the review panel removes the redundancy that cross-review depends on.

4 Implementation

4.1 Agent Architecture

Each agent in Σ is defined by a configuration tuple:

$$A_i = \langle \text{role, goal, backstory, } \mathcal{K}_i, \mathcal{T}_i, \mathcal{M} \rangle \quad (2)$$

where $\mathcal{K}_i$ is a set of domain skill files (Markdown documents injected into the agent’s system prompt), $\mathcal{T}_i$ is the set of tools available to the agent, and $\mathcal{M}$ is the shared LLM backend. Agents do not share memory beyond the explicit context state S_t —there is no implicit “conversation history” that could accumulate errors across rounds.

4.2 Tool Integration

Tools follow a `BaseTool` interface with `name`, `description`, and a `_run` method. Sigma generates OpenAI function-calling schemas from tool definitions at runtime:

```
class BaseTool:
    name: str
    description: str
    def _run(self, **kwargs) -> dict: ...
    @classmethod
    def to_openai_schema(cls) -> dict: ...
```

The framework prioritizes native function calling (LLM dynamically selects parameters) with a text-marker fallback (`[TOOL_NEEDED: tool_name]`) for models without function-calling support.

4.3 Open-Source Release

Σ is released as `sigma-aerc` on PyPI (v0.1.0, <https://pypi.org/project/sigma-aerc/>), with 985 tests and `py.typed` type stubs. The GreenBuild project repository—including all agent definitions, engineering tools, and design iteration history—is available at <https://github.com/maimai-dot/RocketFactory> under AGPLv3.

5 Case Study: GreenBuild Energy Retrofit

5.1 Project Context

The GreenBuild project aims to design a deep energy retrofit for a 15,000 m², 10-story commercial office building originally constructed in 1998 in a cold-climate region (comparable to Beijing, ASHRAE Climate Zone 5A). The building’s aging gas-fired boiler and constant-volume chiller, combined with a single-glazed curtain wall, result in an estimated baseline annual energy use intensity (EUI) of approximately 185 kWh/m²—roughly double the current best-practice target for new commercial offices in the region [11]. Constraints:

- All retrofit measures must use commercially available, non-proprietary equipment
- Total capital budget under **¥3,000,000** (~\$420,000)
- 12-month design and construction timeline
- Building must remain partially occupied during retrofit (phased implementation)
- Target EUI after retrofit: ≤ 110 kWh/m² (approximately 40% reduction)

5.2 HVAC System Analysis

The primary retrofit strategy centered on replacing the aging gas boiler and constant-volume chiller with an air-source heat pump (ASHP) system capable of serving both heating and cooling loads. The Energy Modeler performed an initial EnergyPlus simulation using manufacturer-rated coefficient of performance (COP) values for the candidate ASHP:

System	COP (manufacturer)	COP (seasonal corrected)	Annual Energy Use Intensity (EUI)
Air-source heat pump (ASHP)	5.8	4.75	38 kWh/m ²
Ground-source heat pump (GSHP)	5.2	4.60	44 kWh/m ²
Gas boiler + electric chiller	0.85 / 2.8	0.82 / 2.6	baseline 185 kWh/m ²

Table 3: Heating system performance comparison. Seasonal corrected COP accounts for part-load degradation, distribution losses, and cold-climate capacity derating. The ASHP provides both heating and cooling; gas boiler COP represents annual fuel utilization efficiency.

Error correction. The Energy Modeler’s initial EnergyPlus simulation used manufacturer-rated COP values directly from the heat pump selection software (COP = 5.8 at AHRI 550/590 standard rating conditions). During Phase R cross-review, the HVAC Engineer noted that these values assume full-load operation at mild outdoor air temperatures and do not account for real-world degradation factors. The Code Reviewer, acting as devil’s advocate, questioned whether the model adequately represented worst-case winter conditions. The Building Physicist’s independent review confirmed three compounding effects that had been overlooked:

1. **Part-load degradation (−8%).** The heat pump operates at part load for approximately 70% of annual operating hours. Manufacturer COP is measured at full load only; part-load efficiency—particularly during frequent cycling in shoulder seasons—is systematically lower.
2. **Hydronic distribution losses (−5%).** The existing building uses a hydronic fan coil system with uninsulated risers running through unconditioned shafts. Heat loss from distribution piping adds approximately 5% to system energy consumption and is not captured by equipment-only COP values.

3. **Cold-climate capacity derating (−7%).** At the design heating outdoor air temperature of -12°C , air-source heat pump heating capacity drops by approximately 15% and COP degrades by roughly 7% from the rated condition at 7°C outdoor air temperature.

The compounded effect: actual expected seasonal $\text{COP} \approx 4.75$ —an 18% reduction from the manufacturer catalog value of 5.8. The correction had cascading implications for supplemental electric resistance heating requirements during peak cold days, equipment sizing decisions, and the simple payback calculation (extending from a claimed 6.2 years to a corrected 8.5 years). This finding is consistent with the well-documented performance gap between rated and in-situ heat pump performance reported in field studies of building retrofits [10].

5.3 Envelope and System Design

Parametric modeling via OpenStudio SDK produced a baseline building model and evaluated retrofit measure packages:

- Curtain wall upgrade: single-glazed ($U \approx 5.7 \text{ W/m}^2\text{K}$) to triple-glazed low-e ($U \approx 0.8 \text{ W/m}^2\text{K}$), reducing envelope heat loss by approximately 42%
- Air sealing: reducing infiltration from 0.8 to 0.3 air changes per hour at 50 Pa
- Demand-controlled ventilation (DCV) with CO_2 sensors, reducing outdoor air heating/cooling load during partial occupancy
- Direct digital control (DDC) upgrade with predictive start/stop and supply air temperature reset

Energy savings estimation remains subject to model calibration quality. Early consensus estimation (Section 3.2) yielded baseline EUI of $185 \pm 22 \text{ kWh/m}^2$ before submetered energy data became available. Initial EnergyPlus simulation with default internal load assumptions returned a baseline EUI of 198 kWh/m^2 , while the Building Physicist’s calibrated model using 12 months of utility bills yielded 172 kWh/m^2 —a 15% spread that illustrates the value of Sigma’s multi-path verification: a single-point estimate from any single source would have gone unquestioned.

5.4 Key Design Findings

After multiple design iterations, Σ converged on the following engineering conclusions:

1. **ASHP viability confirmed.** The corrected seasonal COP of 4.75 is sufficient for the cold-climate application with properly sized supplemental electric resistance for the approximately 120 hours per year below -10°C . The GSHP option offers marginally higher savings (44% vs. 38%) but at roughly $2.5\times$ capital cost due to borehole drilling, yielding a payback period exceeding 20 years.
2. **Triple-glazed curtain wall is cost-effective.** The envelope upgrade alone delivers a 42% reduction in conduction heat loss. Cost analysis by the Cost Estimator confirmed that the incremental cost of triple glazing over double glazing is recovered within 9 years through reduced heating energy, making it the single most cost-effective passive measure.
3. **DCV and economizer provide synergistic savings.** The Controls Engineer and Energy Modeler jointly evaluated ventilation strategies. DCV with CO_2 -based control reduces ventilation load by 15–20% during shoulder seasons, while an air-side economizer provides free cooling when outdoor conditions permit. The combined measures yield 22% HVAC energy reduction beyond envelope improvements alone.

4. **Combined package meets the target.** The full retrofit package—envelope upgrade, ASHP replacement, DCV + economizer, and DDC upgrade—reduces annual EUI from the baseline 185 kWh/m² to a predicted 107 kWh/m² (42% reduction), meeting the ≤ 110 kWh/m² target with a 3 kWh/m² margin.

6 Evaluation

6.1 Error Detection

Across the GreenBuild design campaign, cross-review detected 31 substantive issues across RIGOROUS-tier rounds (average 1.7 per round). We classify these into four categories:

1. **Quantitative errors (11; 35%).** Incorrect numerical values—such as the heat pump COP overestimate (Section 5.2)—that would have propagated through a sequential pipeline. These are the highest-severity category because downstream calculations (equipment sizing, energy savings, payback period) compound the error.
2. **Assumption violations (9; 29%).** Cases where an agent applied a model or formula outside its valid range. For example, using steady-state U-value calculations without accounting for thermal bridging at floor slab edges, or applying rated COP at part-load conditions without a part-load degradation factor.
3. **Omitted constraints (8; 26%).** Cases where an agent’s analysis overlooked a hard constraint. Examples include ASHRAE 62.1 minimum ventilation requirements [11], local fire code clearances for outdoor heat pump units, and structural load limits for rooftop equipment placement.
4. **Terminology mismatches (3; 10%).** Cases where agents used the same term for different concepts (e.g., “efficiency” interpreted as COP by the HVAC Engineer, EER by the Controls Engineer, and SEER by the Energy Modeler).

The devil’s advocate mechanism contributed 5 unique findings across categories 1–3 that no other agent identified independently. Notably, the heat pump COP overestimate (category 1) was caught because the Code Reviewer questioned whether manufacturer COP data reflected the actual operating conditions in the specific building context—a line of inquiry that domain specialists, focused on their own analyses, did not independently pursue.

6.2 Convergence Behavior

RIGOROUS-tier tasks converged in 2–3 rounds (median 3, $n = 6$ tasks) within the 4-round limit, with the majority of first-round failures caused by tool data unavailability rather than agent disagreement. One task hit the oscillation detector (agents alternating between ASHP and GSHP for the primary heating system), resolved by the Project Architect breaking the tie based on lifecycle cost analysis at the project’s 15-year planning horizon.

6.3 Cost Efficiency

The complexity-adaptive system reduced costs by approximately $6\times$ for LITE-tier tasks compared to running the full RIGOROUS protocol. A typical RIGOROUS building energy design round consumes $\sim 12,000$ tokens ($\sim \text{¥}0.04$ at DeepSeek V4 Pro pricing), making the economic barrier to iterative engineering design negligible.

6.4 Design Evolution and Negative Results

Several early design choices were revised based on empirical observation. We document these as guidance for future protocol designers:

1. **Sequential review was abandoned.** An early version of the protocol allowed agents to review analyses in a round-robin sequence (Agent 1 reviews Agent 2, Agent 2 reviews Agent 3, etc.). This created an ordering bias: agents reviewed earlier in the sequence received more scrutiny than those reviewed later, when reviewer fatigue and anchoring to earlier verdicts set in. The current pairwise assignment with domain-diverse pairing eliminates this ordering effect.
2. **Open deliberation during convergence is necessary.** We initially experimented with keeping agents isolated through Phase C as well, using only the numerical convergence metric to declare consensus. This failed for tasks where agents disagreed on qualitative trade-offs (e.g., capital cost vs. operational savings) but produced numerically similar estimates. Adding open deliberation in Phase C—where agents can see and respond to each other’s final positions—resolved this, at the cost of reintroducing potential anchoring in the final round. We consider this an acceptable risk because (a) convergence triggers only after at least 2 rounds of blind analysis, and (b) the Project Architect’s tie-breaking authority provides an override.
3. **The 5% convergence threshold is task-calibrated.** Our initial 1% threshold caused unnecessary additional rounds for tasks where tool precision was inherently lower than 1% (e.g., EnergyPlus annual energy simulations, where weather file interpolation and HVAC part-load curve discretization cause inherent variability exceeding 2%). The current 5% threshold was calibrated against the worst-case tool precision observed across all Green-Build tools. For deployments with higher-precision tools, this threshold can be tightened.
4. **Early stopping by the LLM failure detector prevented two silent failures.** In two separate RIGOROUS rounds, API degradation caused 4 of 8 agents to return responses below the 20-token verbosity threshold. Without the automatic abort, the remaining 4 agents’ analyses would have appeared to converge, producing a report based on half the intended review panel. Both rounds were re-run successfully after the API stabilized.

7 Related Work

Multi-agent frameworks. Table 4 compares Σ with three widely-used multi-agent frameworks on the structural properties most relevant to engineering applications.

CrewAI [2] popularized role-based agent orchestration but leaves review and quality assurance entirely to user implementation. AutoGen [3] provides flexible multi-agent conversation patterns including group chats and nested chats, but agents share full conversation context by default—information isolation must be manually configured per workflow. LangGraph [1] offers the most control through explicit graph-based agent topology, but every review edge must be manually defined, making protocol enforcement the user’s responsibility. Σ ’s key distinction is that the AERC protocol is *enforced by the framework*, not left to user discipline.

We emphasize that Table 4 is a structural-feature comparison. Quantitative error-detection benchmarks across frameworks under identical task conditions are not yet available—conducting such a controlled comparison, where the same engineering design tasks are executed on CrewAI, AutoGen, LangGraph, and Σ with identical LLM backends and tool access, would provide stronger evidence for the efficacy of protocol-enforced review and represents an important direction for future work.

Property	CrewAI	AutoGen	LangGraph	Σ (AERC)
Role-based agents	✓	✓	✓	✓
Information isolation	—	—	manual only	✓ (enforced)
Cross-review	—	—	manual only	✓ (automatic)
Devil’s advocate	—	—	—	✓
Consensus estimation	—	—	—	✓
Complexity-adaptive	—	—	—	✓ (3 tiers)
LLM failure detection	—	—	—	✓
Physics sanity checks	—	—	—	✓
Open-source	✓	✓	✓	✓

Table 4: Structural comparison of multi-agent frameworks on properties relevant to engineering design. “manual only” indicates the property can be implemented by the user but is not enforced or automated by the framework.

Multi-agent debate. Du et al. [4] showed that multi-agent debate improves reasoning accuracy, but their approach uses homogeneous agents debating the same prompt rather than domain-specialized agents with complementary expertise.

AI for engineering. Recent work explores the application of LLMs to engineering design workflows [5], but typically deploys them in single-agent or simple sequential settings without structured quality assurance. Σ contributes a protocol layer that adds adversarial review and consensus mechanisms without requiring domain-specific model fine-tuning.

Consensus mechanisms. Distributed consensus algorithms such as Paxos [8] and Raft [9] address agreement on discrete values in distributed systems. Σ ’s consensus estimation adapts the idea of structured agreement to continuous engineering estimates with calibrated uncertainty.

8 Limitations and Future Work

1. **LLM dependency.** Σ inherits the reasoning limitations of its underlying LLM. While the protocol catches many errors, it cannot compensate for fundamental failures in physical reasoning. Future work should explore hybrid neuro-symbolic approaches where physics simulators serve as ground-truth validators.
2. **Tool reliability.** Several GreenBuild tools (EnergyPlus weather file sensitivity, building geometry import from legacy CAD files) produced divergent results across model versions. The consensus estimation mechanism partially compensates, but calibrated simulation data would improve convergence quality.
3. **Single-domain validation.** The current evaluation is limited to building energy engineering. Cross-domain validation (e.g., civil infrastructure, drug discovery, legal reasoning) is needed to establish generality. We hypothesize that the AERC protocol’s error-detection properties should transfer to any domain where (a) errors are costly, (b) multiple specialized perspectives exist, and (c) ground-truth verification is possible but expensive—but this hypothesis remains untested.
4. **Human-in-the-loop.** Σ currently places the human (Founder) as an external decision authority. Tighter integration—where the human participates as a first-class agent in the AERC cycle—could improve both safety and output quality for high-stakes applications.
5. **Scaling.** The 8-agent configuration has not been tested at larger scales (20+ agents). Cross-review overhead grows as $O(n^2)$ in the number of agents, suggesting a need for hierarchical review structures in larger deployments.

6. **When not to use AERC.** The protocol’s redundancy-over-efficiency design is counterproductive for tasks where (a) errors are inconsequential (e.g., brainstorming, creative writing), (b) real-time latency dominates accuracy concerns (e.g., live dialogue systems), or (c) the task is a single-step lookup that LITE-tier direct execution handles adequately. Users should assess whether the cost of independent multi-agent analysis is justified by the consequences of an undetected error in their specific domain.

9 Conclusion

We presented Sigma (Σ), an AERC protocol-based multi-agent framework that introduces structured independent analysis, mandatory cross-review, and complexity-adaptive tiering to address three core failure modes of current multi-agent LLM systems: error propagation, groupthink convergence, and cost-rigidity. Through the GreenBuild case study—a real-world design exercise for a 15,000 m² commercial building deep energy retrofit—we demonstrated that protocol-enforced adversarial review can catch physically significant errors that escape both single-agent and sequential multi-agent analysis.

The central finding is that *information isolation during analysis*—preventing agents from seeing each other’s outputs until the Review phase—is a lightweight but effective structural defense against anchoring bias and premature consensus. This suggests that multi-agent reliability can be improved through protocol design alone, without requiring model fine-tuning, reward modeling, or larger models.

Sigma is open-source (AGPLv3), extensively tested (985 tests, 80% coverage), and available as `sigma-aerc` on PyPI. While the framework inherits the reasoning limitations of its underlying LLM and has been validated in a single engineering domain to date, we believe the AERC protocol is general. We specifically invite the community to replicate these findings in other high-stakes domains—civil infrastructure, drug discovery, legal reasoning—where error propagation through multi-agent systems carries real-world consequences.

Data and Code Availability

All source code, agent definitions, engineering tools, design iteration records, and raw output logs of the GreenBuild project are publicly archived at <https://github.com/maimai-dot/RocketFactory> under the AGPLv3 license. The Sigma framework is released independently as the `sigma-aerc` PyPI package (v0.1.0, <https://pypi.org/project/sigma-aerc/>) with 985 tests and full type annotations. The complete design history—including per-round agent analyses, review verdicts, convergence decisions, and tool outputs—is preserved in the repository’s `output/` directory as versioned `REPORT.md` and `result.json` files. This paper was compiled from the same public repository; readers can verify every numerical claim against the archived design records.

Acknowledgments

This work would not have been possible without the open-source engineering software ecosystem. We specifically acknowledge: the U.S. Department of Energy’s Building Technologies Office for developing and maintaining EnergyPlus—the gold standard for whole-building energy simulation since 2001—and the OpenStudio coalition (NREL and collaborators) for their open-source SDK that makes parametric building energy modeling accessible; ASHRAE for maintaining technical standards that form the backbone of building performance engineering; and the broader building performance simulation community for decades of model validation research. The Sigma framework was developed using DeepSeek V4 Pro as the primary LLM backend. The first author

conceived and directed the GreenBuild project, designed the AERC protocol, and wrote the Sigma framework implementation.

References

- [1] LangChain, “LangGraph: Build Language Agent Workflows,” <https://github.com/langchain-ai/langgraph>, 2024. Accessed: May 2026.
- [2] CrewAI, “Multi-Agent Framework,” <https://github.com/crewAIInc/crewAI>, 2024. Accessed: May 2026.
- [3] Wu, Q. et al., “AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation,” *arXiv:2308.08155*, 2023.
- [4] Du, Y. et al., “Improving Factuality and Reasoning in Language Models through Multiagent Debate,” *arXiv:2305.14325*, 2023.
- [5] Bommasani, R. et al., “On the Opportunities and Risks of Foundation Models,” *arXiv:2108.07258*, 2021.
- [6] Tversky, A. and Kahneman, D., “Judgment under Uncertainty: Heuristics and Biases,” *Science*, Vol. 185, No. 4157, pp. 1124–1131, 1974.
- [7] Janis, I.L., *Victims of Groupthink: A Psychological Study of Foreign-Policy Decisions and Fiascoes*, Houghton Mifflin, 1972.
- [8] Lamport, L., “The Part-Time Parliament,” *ACM Transactions on Computer Systems*, Vol. 16, No. 2, pp. 133–169, 1998.
- [9] Ongaro, D. and Ousterhout, J., “In Search of an Understandable Consensus Algorithm,” *Proc. USENIX ATC*, pp. 305–319, 2014.
- [10] U.S. Department of Energy, “EnergyPlus Version 24.1.0: Engineering Reference,” <https://energyplus.net/>, 2024. Accessed: May 2026.
- [11] ASHRAE, “ANSI/ASHRAE Standard 90.1-2022: Energy Standard for Buildings Except Low-Rise Residential Buildings,” American Society of Heating, Refrigerating and Air-Conditioning Engineers, 2022.